MID-1

2. phases of NLP.

NLP works is diff phases where each phase handles a diff part of understanding human language.

lexical & morphological Analysis → Syntactic Analysis → Semantic Analysis → Disclosure Integration
↓
Pragmatic Analysis

1] lexical & morphological.

lexical : focuses on words & how the appear in text.

steps 1] Breaks sentances into tokens
2] Identifies parts of speech.
3] Recognizes puntuations, number & symbols.

Eg: "I am studying" ⇒ Tokens = {I, am, studying}.

2 Morphological
focuses on structure of words
steps 1] Breaks words into morphems.
2] performs stemming & lemmatization.

Eg: unbreakable → un + breakable.

__/__/____

2] Syntactic ~~par~~Analysis.

Checks the grammer structure of sentance.

steps : 1] checks the sentance is grammatically correct or not.

2] Builds a parse tree.

3] Detects subjects, verbs, objects, phrases.

tasks: POS tagging, Ambigvity resolution, Building syntax trees.

Goal: Understand how words combine to form meaningful sentances.

3] Semantic Analysis.

Tries to understand the meaning of the sentance.

steps 1] Assigns meaning to words or sentances.

2] Handles word sense.

3] Resolves semantic ambiguities.

4] performs name entity recognition (NER)

tasks: Word sense Disambiguation

Identifying entities like name, place, date, money.

Eg: "Apple looking to BY a startup"

Apple = Company, not fruit.

4] Disclosure Integration.

This ensures that the sentance makes sense with prev sentances.

steps 1] handles Anaphora : "he", "she", "it", "they".

2] Maintains Conten across sentances

3] Builds ~~content bw~~ logical connections blw sentances.

Eg: Rohith went home, he was tired

He refers to rohith. → DI solves this

5] Pragmatic Analysis:

focuses on real world meaning & speakers intention.

- 1] understands intended meaning
- 2] uses real world examples.
- 3] Identifies tone, politeness, requests, commands.

Eg: "Can you pass the salt?"

→ literally a question but pragmatically a request

3. Applications of NLP.

1] Voice Assitants:

Google, Siri, Alexa use NLP to understand the user's spoken comands.

- Speech → text → meaning → action.

2] Text Analysis:

Extracting useful info from text.

used in; Reviews, social media monitoring, feedback analysis

3] Information Retrieval:

helps find relevant info from large collections of data

4] Speech to text:

Spoken language → written text.

5] Chatbots.

Understand user question and give correct responses.

6] Content Recommendation.

Understand what user likes by analyzing prev serches, tent history, watch history etc.

__/__/____

9. Morphemes:

Smallest meaningful unit of a word.
If you break it further, the meaning will be changed or lost.

Eg: Dogs → Dog + s unhappy → un + happy.

- Free morphemes
  - Can stand alone as complete words.
  - Eg: book, kind, un.
- Bound morphemes:
  - Cannot stand alone, must attach to another morpheme.
  - Eg: -s, un-, -ness, -ed.
  - Prefix
    - Attached before a root word
    - Eg: unhappy, redo
  - Suffix
    - after root word.
    - Eg: cats, kindness.
  - Infix.
    - → inside a word
    - → mostly not seen in English

10 TOKENIZATION:

① Word tokenization
- → splitting a sentence or text into individual words. Called "tokens".
- → language dependent.
  - Eg: don't can be [do, n't] or [don't] based on tokenizer

② character tokenization.
- → splitting text into individual characters, rather than words or subwords.
  - Eg: [hello] = ['h', 'e', 'l', 'l', 'o']

Sub Word tokenization :

- Splits text into smaller units than words but larger than characters.

  Eg: "unhappiness" → un + happi + ness.

Methods :

method	Description	tools / Models.
Byte pair encoding (BPE)	Merges freq. pairs of chars / subwords	GPT / ROBERTa
Word Piece.	Greedy longest match subword segmentation	BERT.
Unigram language model	probabilistic subword selection	XL net, Sentance Piece

Sentence tokenization (or) sentance segmentation.

Splitting a pragraph or text into individual Sentances.

N Gram tokenization

Splits text into overlapping sequence of N items.

N=1 → unigram
N=2 → Bigram
N=3 → Trigram.

Types 1] Word Ngrams → based on word sequence
"New York city" → Unigram - "New" "York" "city"
Bigram - "New York", "York city".
2] Character Ngrams → based on char sequence
"hello", 2grams : 'he', 'el' 'll', 'lo'

__/__/____

4. top down and bottom up parsing.

parsing refers to the process of analyzing and understanding the structure of a sentance written or spoken in human language.

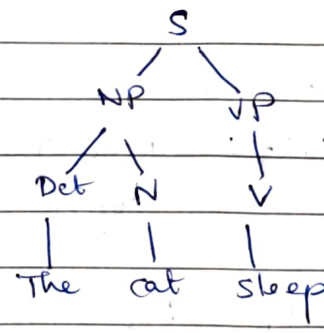
top down: start from the root (S)
and expand until you reach the input words

1] Begin with S
2] Apply production rules to expand non terminals.
3] At each step, choose a rule that moves you closer to matching the input.
4] Stop when the sequence of terminals match the input.

Example (The cat sleeps)

Grammer

S → NP VP
NP → Det N
VP → V
Det → The
N → cat
V → sleeps



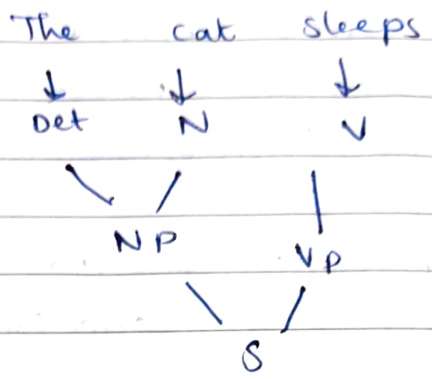
Bottom-Up parsing

1] Begin with input tokens.
2] look at right side of production rule in the current sequence and replace it with left-hand side (reduction)
3] Repeat until the entire input reduces to the start symbol S.

Eg: The cat sleeps

Grammer:

S → NP VP
NP → Det N
VP → V
Det → The
N → cat
V → sleeps



1. Shift Reduce Parsing.

→ It is bottom up parsing technique.
→ parser uses 2 data structures
- Stack → holds symbols
- Input buffer → holds the remaining input tokens.

4 Actions..

Shift: Move the next input token to the top of the stack.

Reduce: Replace a sequence of symbols on the stack that matches the RHS of a grammer rule with LHS.

Accept – If the stack contains the start symbol and input is empty → succesful parse.

Error – If no valid move exists.

__/__/____

Example: $S \rightarrow S+S$
$S \rightarrow S*S$
$S \rightarrow id$

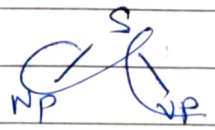
"id + id + id"

stack	Input buffer	Parsing Action.
$	id+id+id$	shift.
$ id	+id+id$	Reduce S → id
$ S	+id+id$	shift
$ S +	id+id$	shift
$ S+id	+id$	Reduce S → id
$ S+S	+id$	Reduce S → S+S
$ S	+id$	shift
$ S +	id$	shift
$ S+id	$	Reduce S → id
$ S+S	$	Reduce S → S+S
$ S	$	Accept.

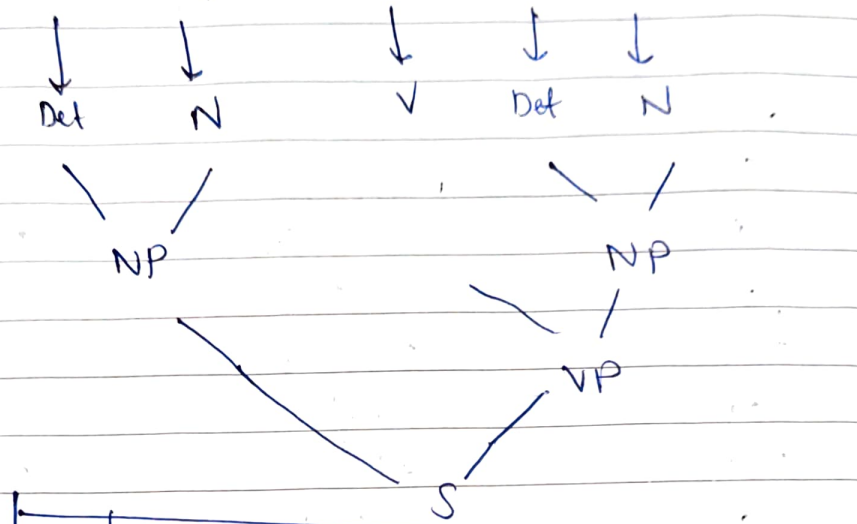
9. CYK parsing. (Cocke-Younger-Kasami Parsing)
→ Bottom up parsing algo used for content free grammer (CFG)
→ It works only if the grammer is in chomsky Normal form (CNF)

Sentance the flight includes a meal.

$S \rightarrow NP|VP$ 0.80
$NP \rightarrow Det|N$ 0.3
$VP \rightarrow V|NP$ 0.20
$V \rightarrow$ include. 0.05
Det → The 0.4
Det → a 0.4
N → meal 0.01
N → flight 0.02



the flight includes a meal



	1	2	3	4	5
0	(0.4) Det ←	NP ←			S
1		N (0.02)			
2			V (0.05) ←		VP 0.0001
3				Det ← (0.4)	NP (0.0012)
4					N (0.01)

$(0,2)\ NP = 0.3(0.4) * (0.02)$

$= 0.0024$

$(3,5)\ NP = 0.3\,(0.4)(0.01)$

$= 0.0012$

$(2,5)\ VP = (0.05)(0.0012)\,0.2$

$= 0.0000012$

$(0,5)\ S = (0.0024)(0.0000012)(0.8)$

$= 0.00001\,92$

__/__/____

5. Lemmatization techniques : process of converting a word into its meaningful base form (lemma)

1] Rule Based Lemmatization : Eg: Caring → care.
→ uses hand written linguistic rules.
→ checks suffixes like ing, ed, s, es.
→ Also uses POS tags
Eg: Running → removing ing → "run".

follows grammer rules to find the correct base form.

2] Dictionary Based Lemmatization :
→ uses predefined dictionary/lexicon of word forms.
→ The system looks up the word in the dictionary and return its lemma.
Eg: better → Dictionary maps → "good".

It searches in the dictionary to find the true root word.

3] Statistical & Machine Learning Approaches :
→ uses trained models like CRF, HMM or neural networks
→ learns patterns from large corpora.
→ Can handle ambeguous words using context.
Eg: "Saw"
noun? → tool
verb? → past of see
ML model chooses based on Sentances.

The computer learns how to find lemmas by studying examples.

4] Hybrid Approaches:

→ combins rules + dictionary + ML.
→ used in advanced NLP tools like Stanford CoreNLP
→ Gives most accurate results.

Mix of all techniques → more power, fewer mistakes

8. Ambiguity Reduction techniques in NLP.

1] contentual Analysis
→ uses surrounding words and sentance context.
→ helps ~~redu~~ identify which meaning fits best.
→ Reduces lenical & pragmatic ambiguity.

2] word sense Disambiguation (WSD)
→ special technique to choose the correct ~~sentance~~ sense of a word with multiple meanings.
Eg: Bank → river bank / money bank.
→ Reduces lenical ambiguity.

3] Parsing and syntactic Analysis.
→ Builds parse treel.
→ Identifies correct grammer structure
→ Reduces syntactic ambiguity.

4] ~~Confer~~ Coreference Resolution
→ finds what pronouns refer to
Eg: "Ravi met Arjun. He was excited" He → Ravi/Arjun.
→ Reduces referential ambiguity.

__/__/____

5] Discourse and pragmatic Modeling
→ uses speaker intent, tone and real world knowledge.
→ Reduces pragmatic ambiguity.

6] Machine learning & Deep learning Models (BERT, GPT)
→ Learns pattern from huge data sets.
→ Use context aware embeddings.
→ Understand ambiguity better than rule based systems.
→ Reduces all types of ambiguities.

6. Stemming Techniques.

"Stemmers might not always result in semantically meaningful base words".

1] porter stemmer
→ It removes common english suffixes in multiple steps
→ uses a well defined rules to strip endings like -ing, -ed -ly.
→ It cuts off common endings using a fixed set of rules.

2] lovins stemmer
→ longest suffix is remove from the word and the remaining stem is adjusted or modified to form a valid word.
Eg: running → -ing → runn → run.

3] Dawson Stemmer.
→ improved version of lovins.
→ In this suffixes are stored back word (ing → gni)
→ The stemmer organizes these reversed suffixes based on → length
→ the last char.
→ Uses reversed tables to stem faster & more accurately.

__/__/____

4] Krovetz stemmer

→ convert the plural form of a word to its singular form.

→ Convert the past tense of a word to its present tense and remove the suffix 'ing'.

Eg: children → child.

5] N Gram stemmer.

→ Breaking words into segments of length N and then applying statistical analysis to identify patterns.

Eg: 'SONS' for n=2 "S, SO, ON, NS, S".

6] Snowball Stemmer.

→ Advanced and improved version of porter stemmer.

→ Supports multiple language.

→ Developed using the Snowball language.

7] lancaster Stemmer.

→ very aggresive iterative stemmer with many stripping rules.

→ often chops toomuch of the word.

→ not as efficient as snowball stemmer.